Transformer -- 学后总结

张彦博

目录

背景 -- background

- FNN
- RNN
- Encoder-Decoder
- Attention

transformer模型

- 整体输入输出
- Encoder-Decoder
- Multi-Head Attention
- Masked Multi-Head Attention

学习思路：
transformer创新点：Transformer架构+self-attention自注意力机制
1. 针对“翻译问题”，以往模型是怎么做的？他们有什么问题？ --- 背景
(对应原论文Introduction+background)
2. 论文提出的transformer架构，他是怎么运行的？同时这样做解决了什么问题？ -- transformer模型
(对应原论文model architecture ＋ why self-attenetion)

Sequence Transduction Model 序列转导模型

- 什么是序列转导模型（Sequence Transduction Model）:是处理序列数据的模型

  序列数据：具有顺序关系的数据，每个元素的顺序对于数据的整体含义非常重要

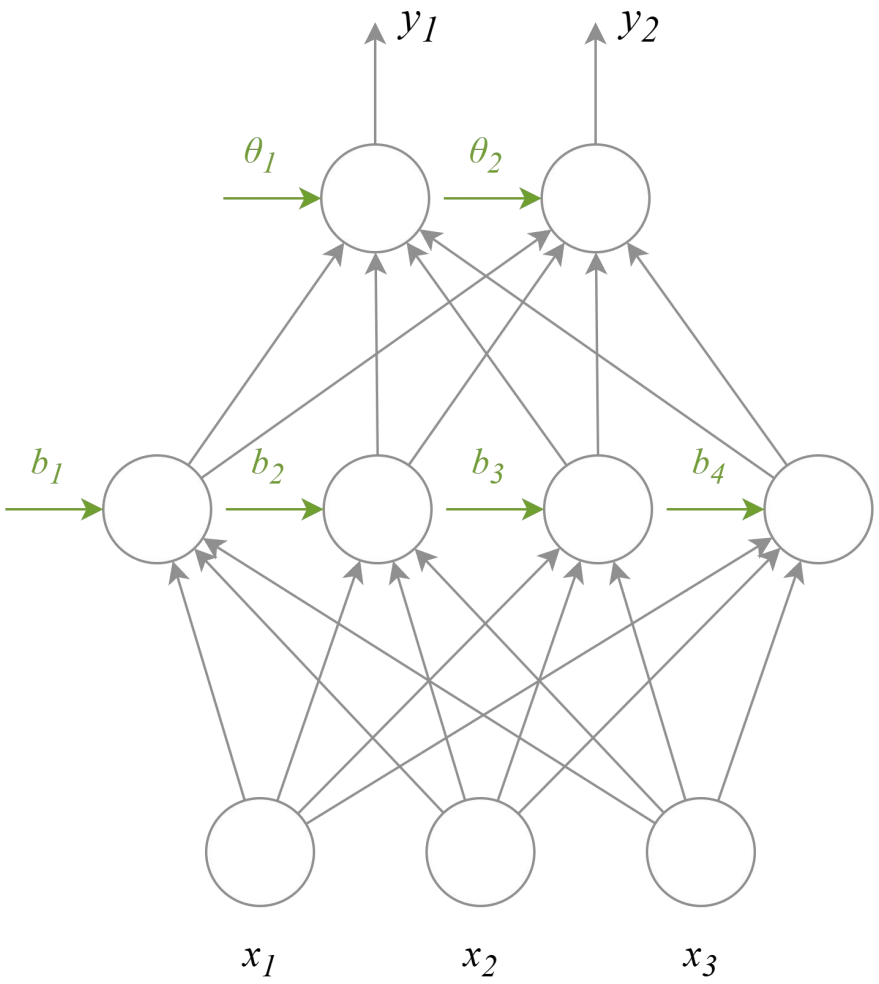
  典型的序列转导任务

  文本翻译：翻译前的文本（"How are you"）→ 翻译后的文本（"你好吗？"）；

  文本生成：用户的输入文本（"How are you?"）→ AI 生成的回复文本（"I'm fine, thank you."）；

  语音转文字：一个音频片段 → 音频的文字转写；

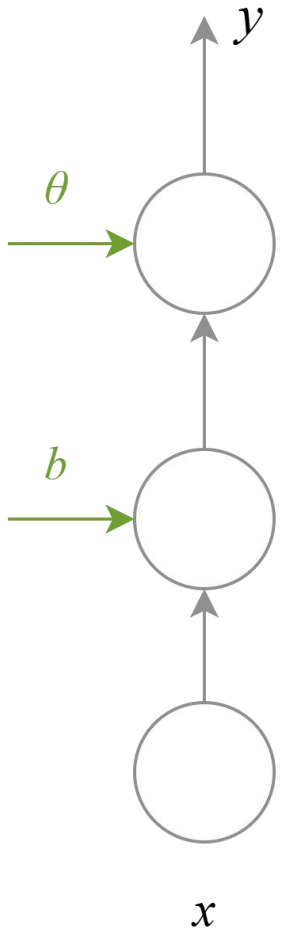
1.FNN



输出层

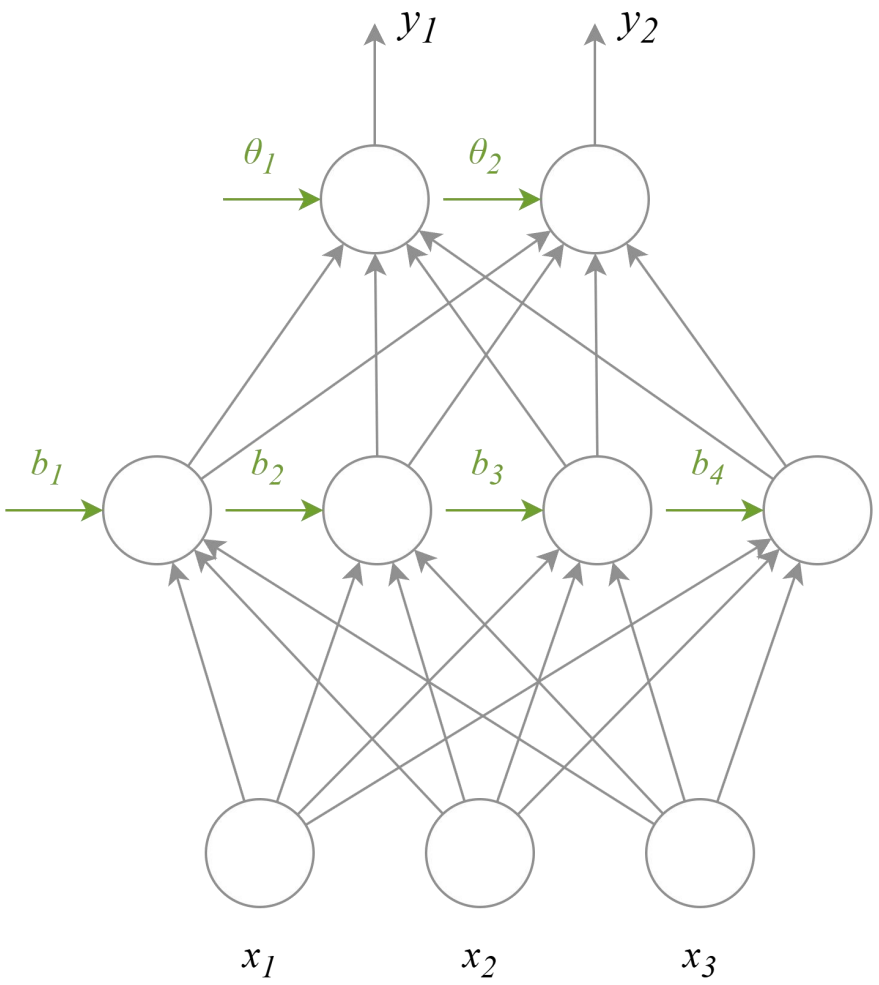
隐藏层

输入层



Feedforward Neural Network:前馈神经网络。信息单向传播，从输入层→隐藏层→输出层，没有循环反馈

1.FNN



- FNN在序列转换任务中的步骤(推理阶段):

输入:我喜欢你

1. 分词（Tokenization）

   ['我','喜欢','你']

2. 语义化表示(Embedding)

   '我'->[0.1,0.5,0.3]

   '喜欢'->[0.2,0.4,0.1]

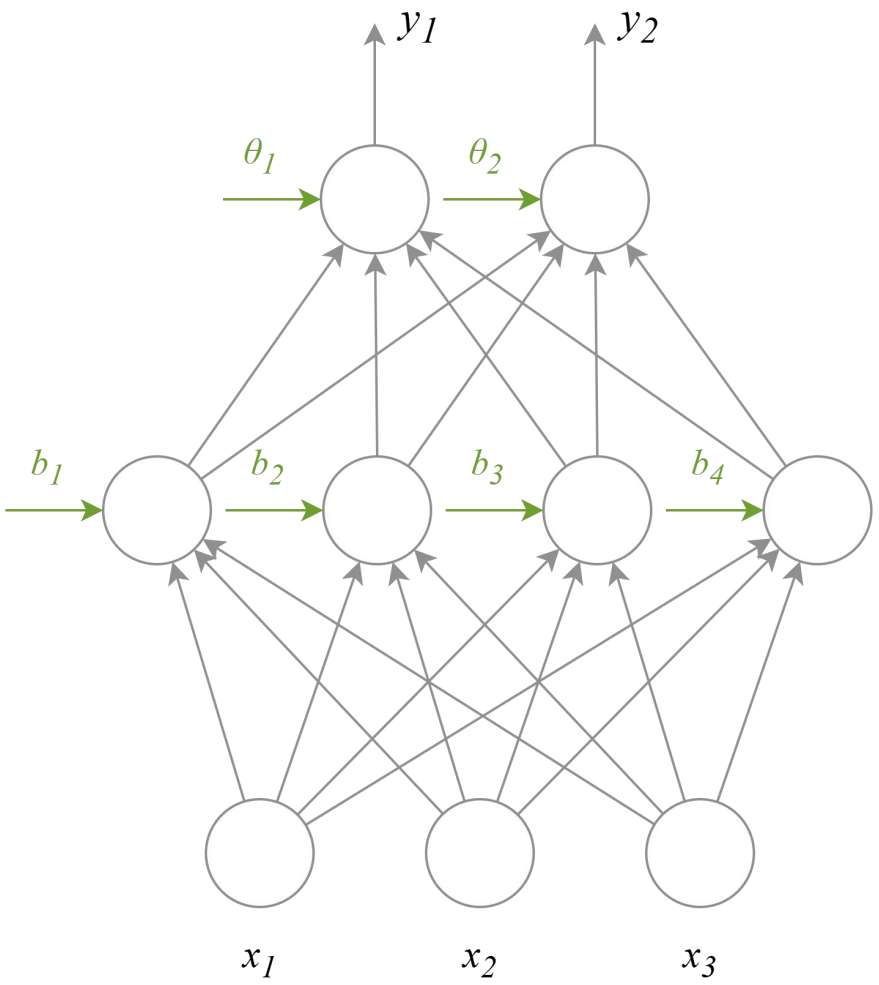
   '你'->[0.7,0.5,0.6]

3. 合并词向量

   平均：词语顺序丢失

   拼接：固定长度输入，句子长度不可能永远相同

1.FNN

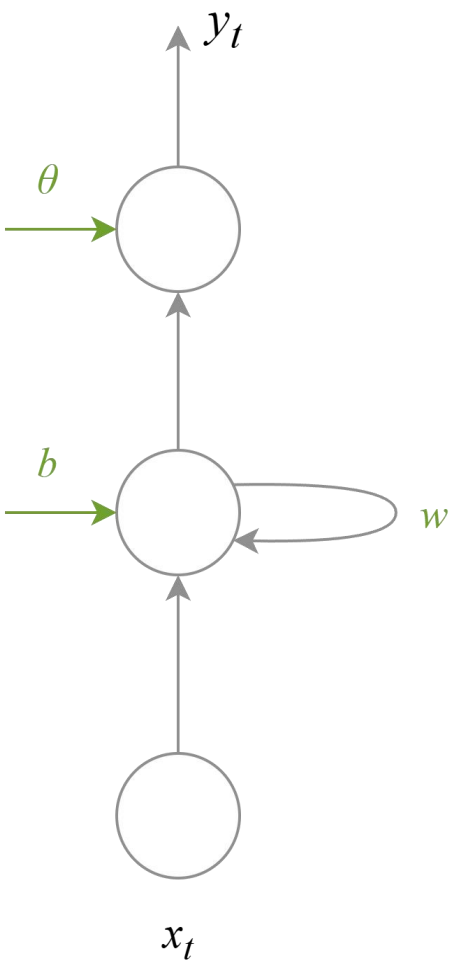


- FNN在序列转换任务中的步骤(推理阶段):

输出:从词汇表中选择一个概率最大的词语

这种不能自回归的结构导致FNN先天不适合做序列任务

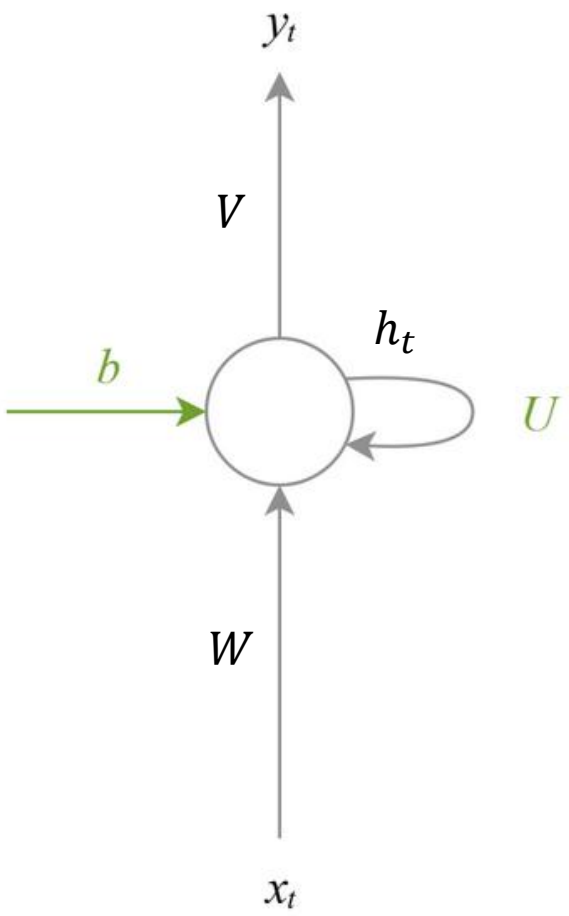
2.RNN



- 能够建模词序：RNN 是按时间顺序（token 顺序）逐个处理输入的；
- 能够建模上下文依赖：RNN 是逐个喂入词语，并且会有“记忆”机制；
- 支持不定长输入：不再需要 FNN 那种固定长度的输入格式，句子多长都行；

RNN:Recurrent Neural Network。循环神经网络。带有循环连接，能保存历史信息，拥有记忆能力

2.RNN:单个循环结构单元

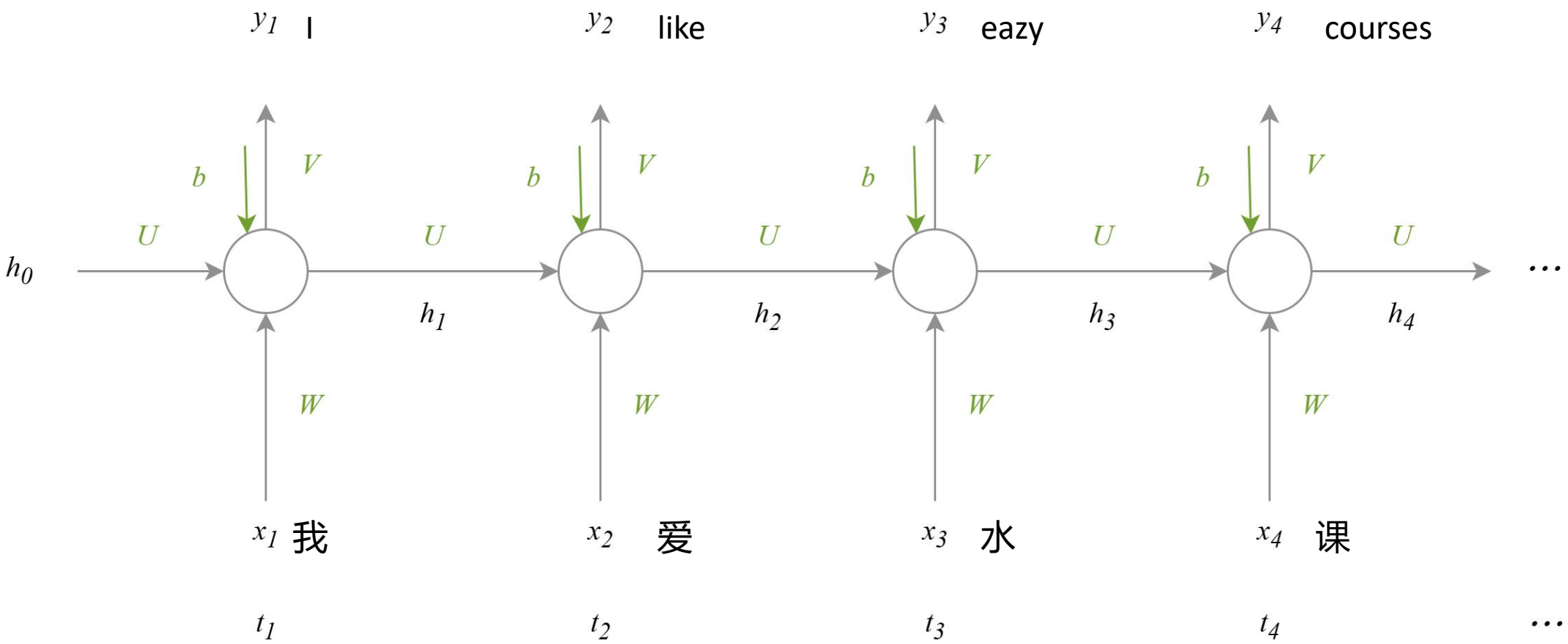


- $h_t = g(Wx_t + Uh_{t-1} + b)$
- $y_t = g(Vh_t)$

x_t：t时间步的输入
h_t：t时间步的隐藏状态；
h_{t-1}：上一时间步隐藏状态（循环带回的记忆）
W：输入x_t对应的权重矩阵
U：隐藏状态循环自环的权重矩阵
b：偏置项
g：激活函数（ReLU/sigmoid 等）

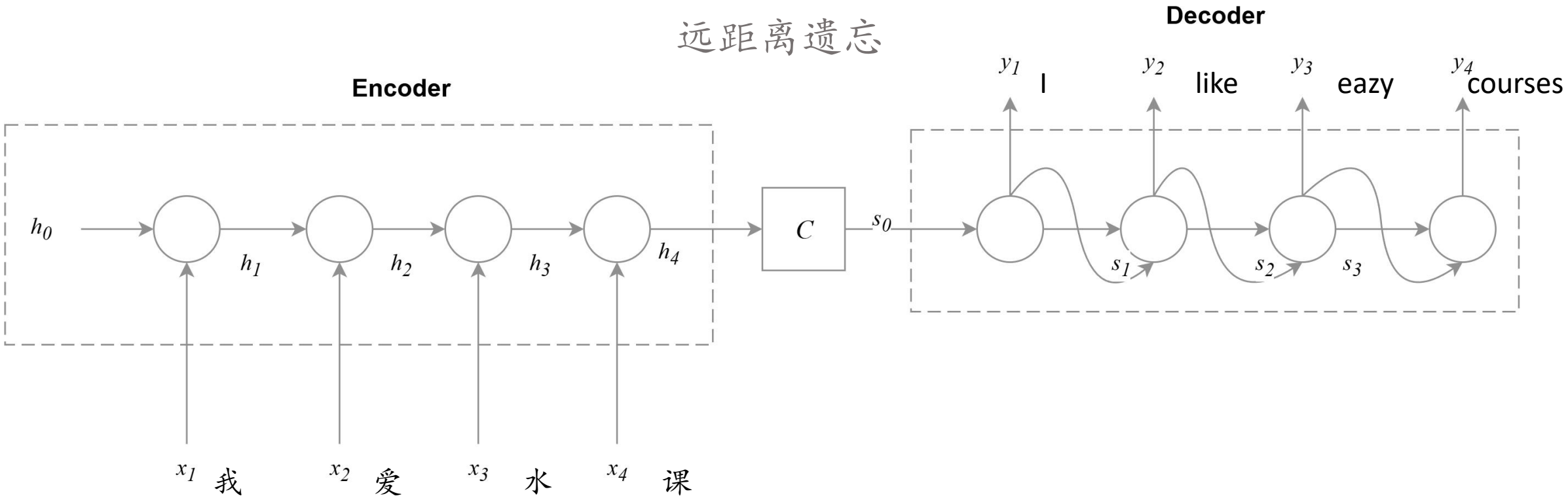
2.RNN

$$h_t = g(Wx_t + Uh_{t-1} + b)$$
$$y_t = g(Vh_t)$$



1. 上下文关系
2. 串行化计算

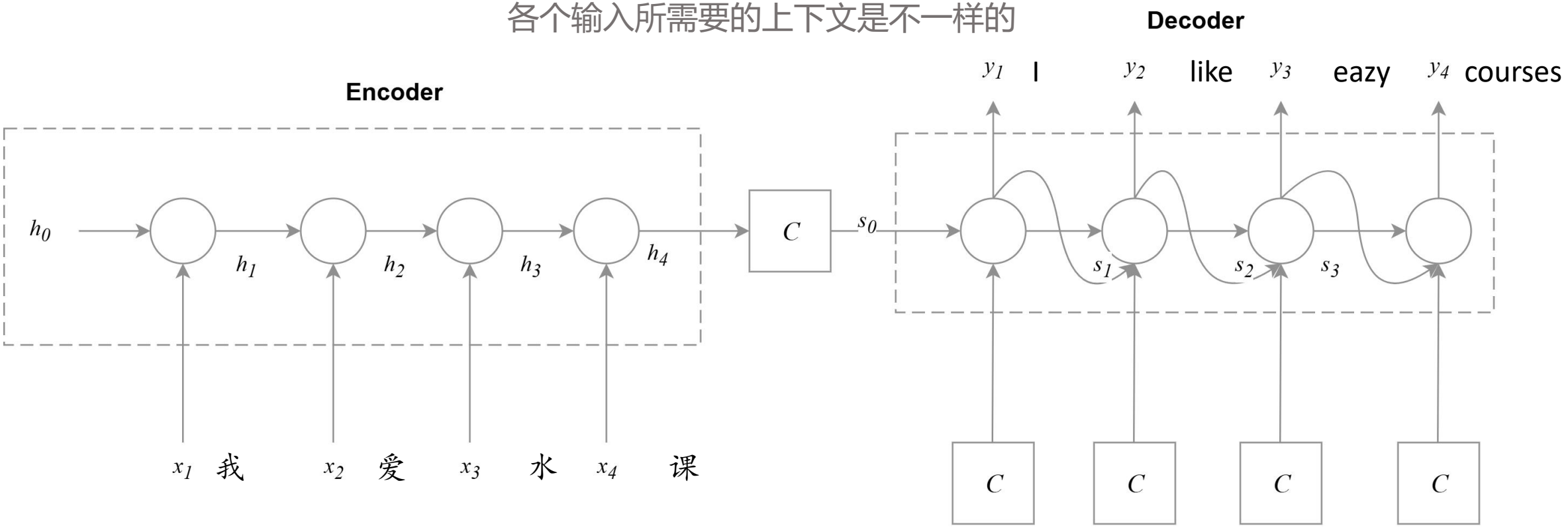
3. Encoder & Decoder



上下文向量C：编码器读完整个输入句子后，把全部语义压缩成的固定长度向量，传给解码器用于生成翻译。

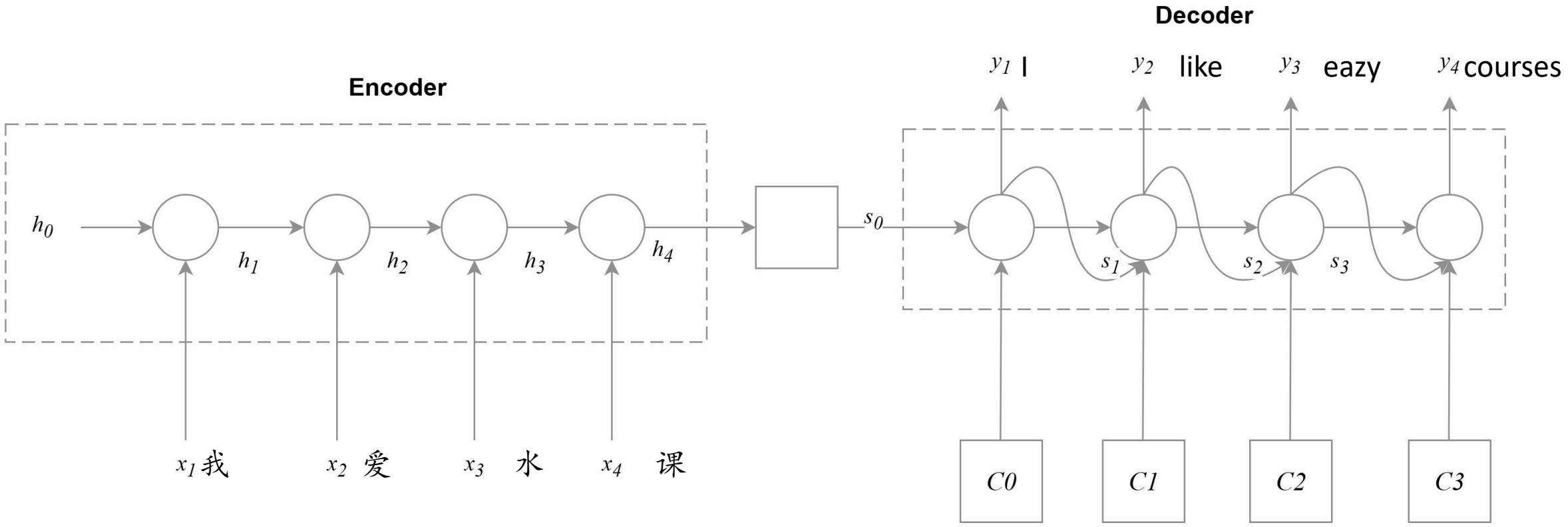
3. Encoder & Decoder

各个输入所需要的上下文是不一样的



方法：输入上下文向量，用于对抗远距离遗忘

4. Attention mechanism注意力机制



$$C_i = \sum_{j=1}^{n} \alpha_{ij} h_j$$

可训练参数

$$C_2 = 0.1h_1 + 0.1h_2 + 0.4h_3 + 0.4h_4$$

5.总结

- FNN:

问题：不能自回归，没有顺序。

- RNN:

问题：上下文、串行化

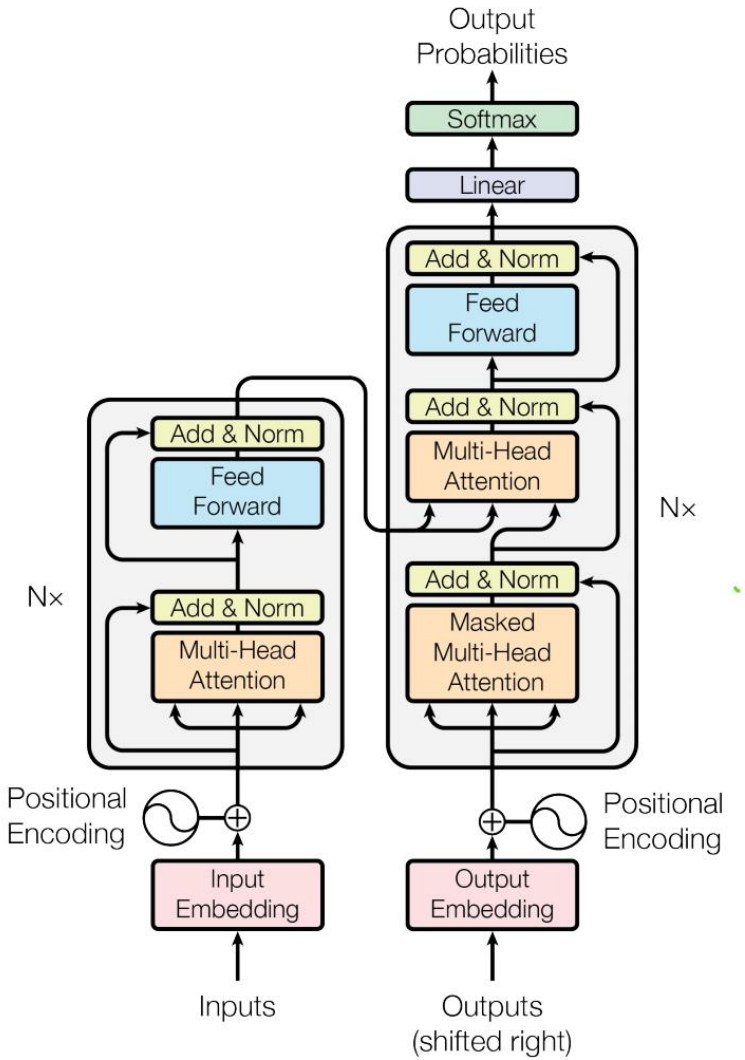
- E&D

问题：翻译时，各个节点对所需要的上下文信息各不相同、串行化

- 注意力机制

问题：串行化

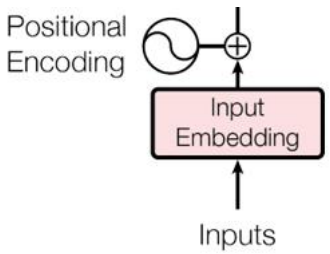
6.Transformer

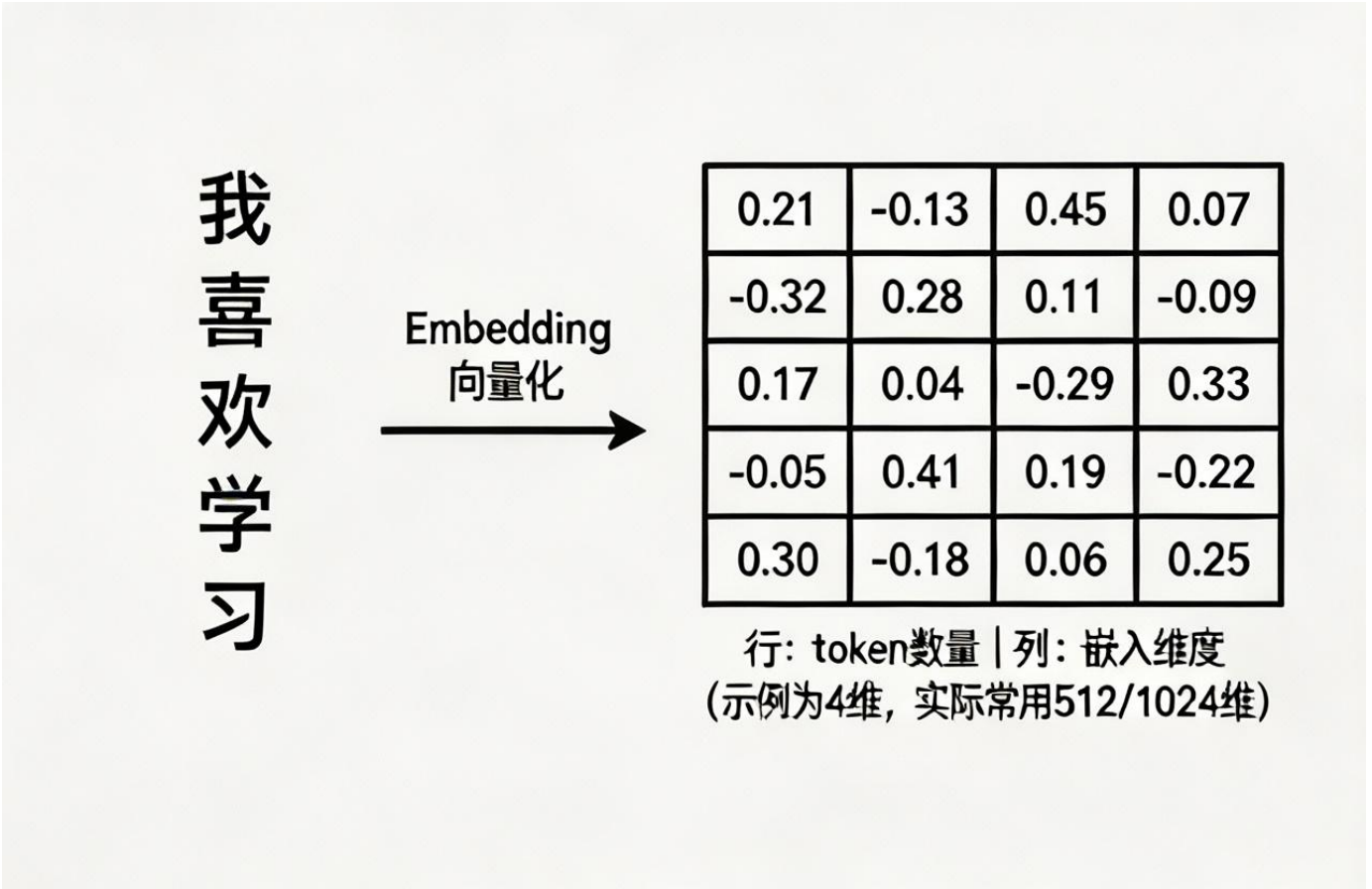


1. embedding & 位置编码
2. 整体输入输出 --> 训练 / 推理
3. 单头自注意力机制
4. 单头掩码自注意力机制
5. 多头自注意力机制

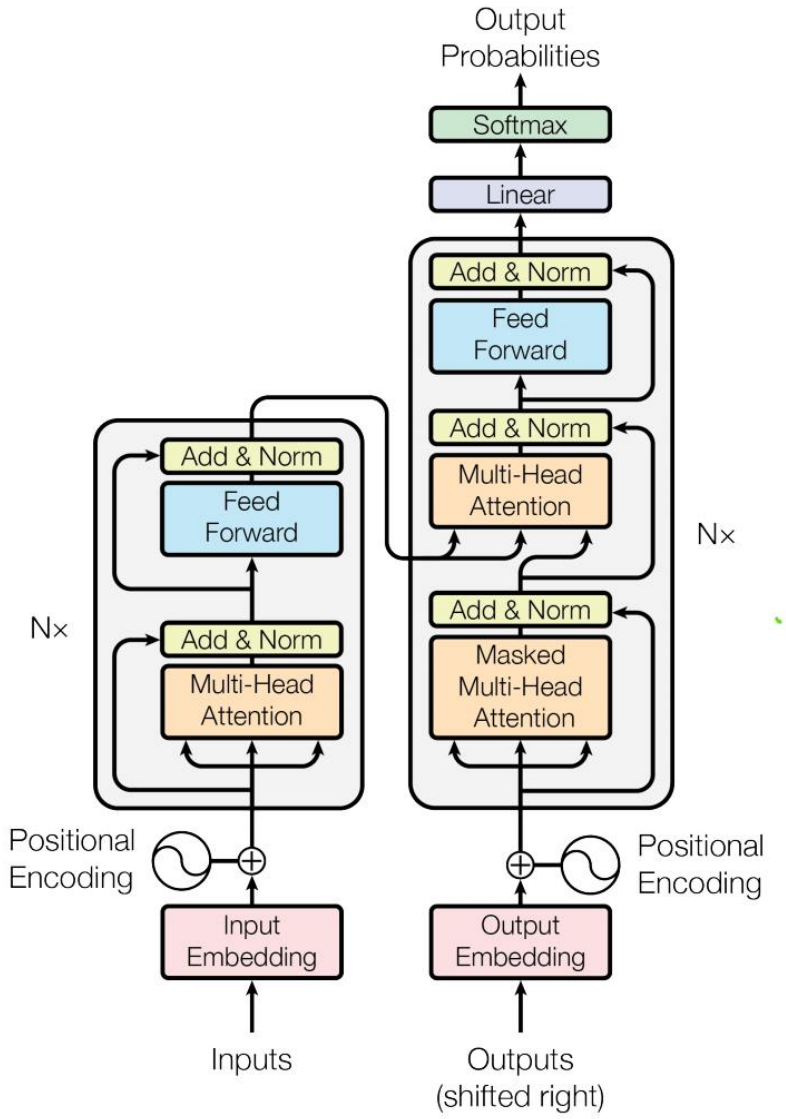
6.Transformer -- 整体输入输出

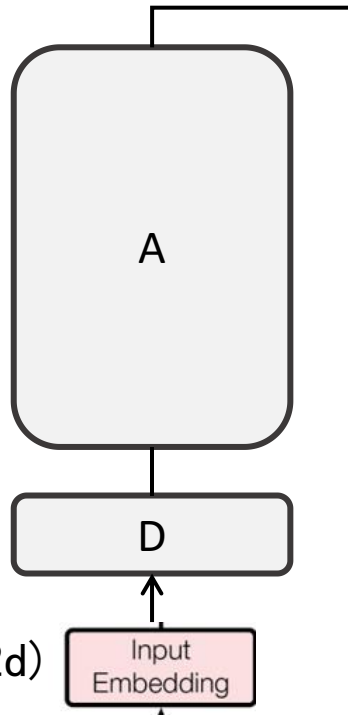
- Embedding
  - 分词：Tokenization
  - 向量化:
  - 拼接成矩阵:
- Positional Encoding:
  - 位置编码
  - 给每一个向量打上Tag，标识其顺序





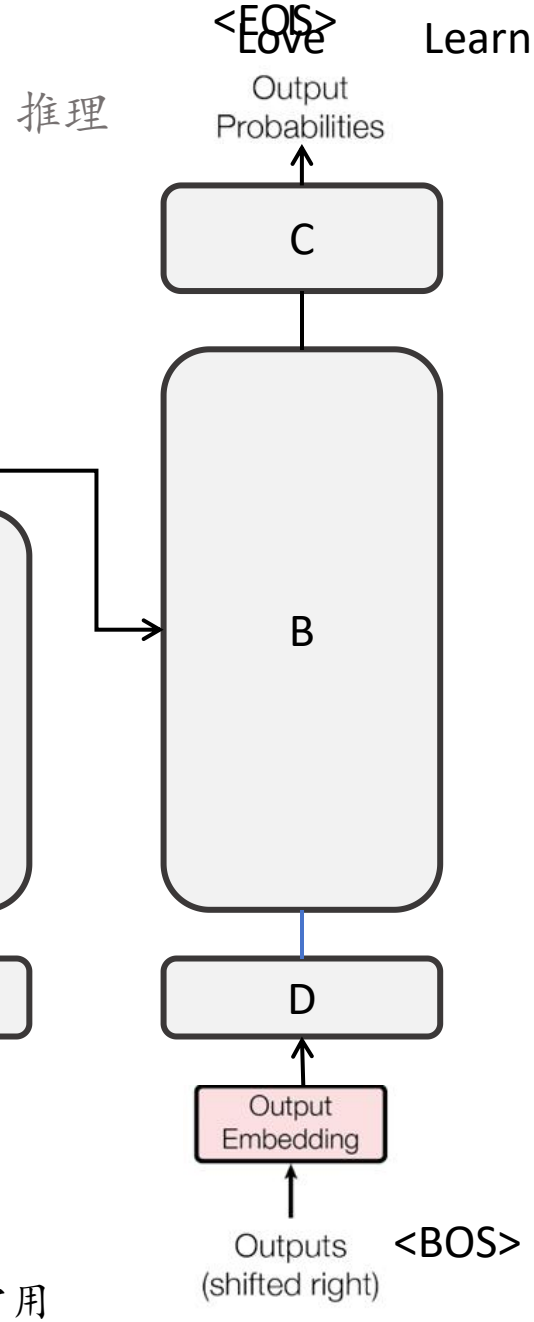
6.Transformer -- 整体输入输出





我爱学习(向量化--512d)

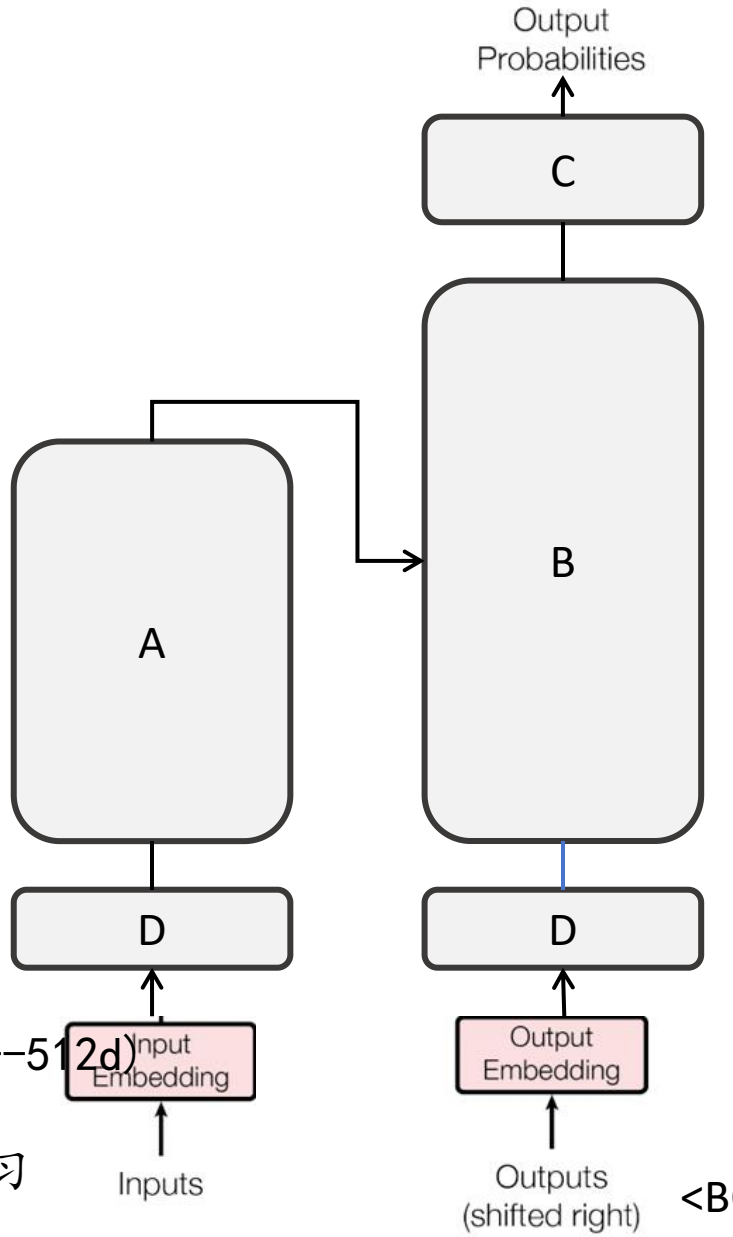
我爱学习



Output Probabilities:下一个位置，词表中每个词 / 字出现的概率 模块C：分类模型常用

6.Transformer -- 整体输入输出

训练

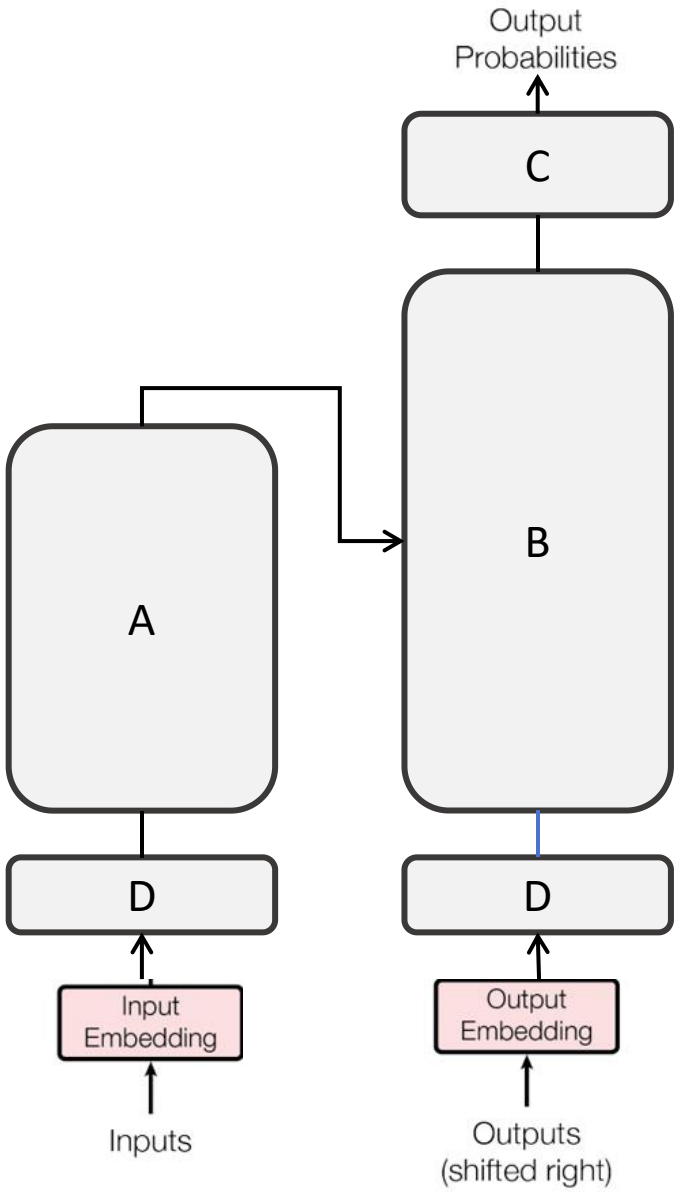


我爱学习(向量化--512d)

我爱学习

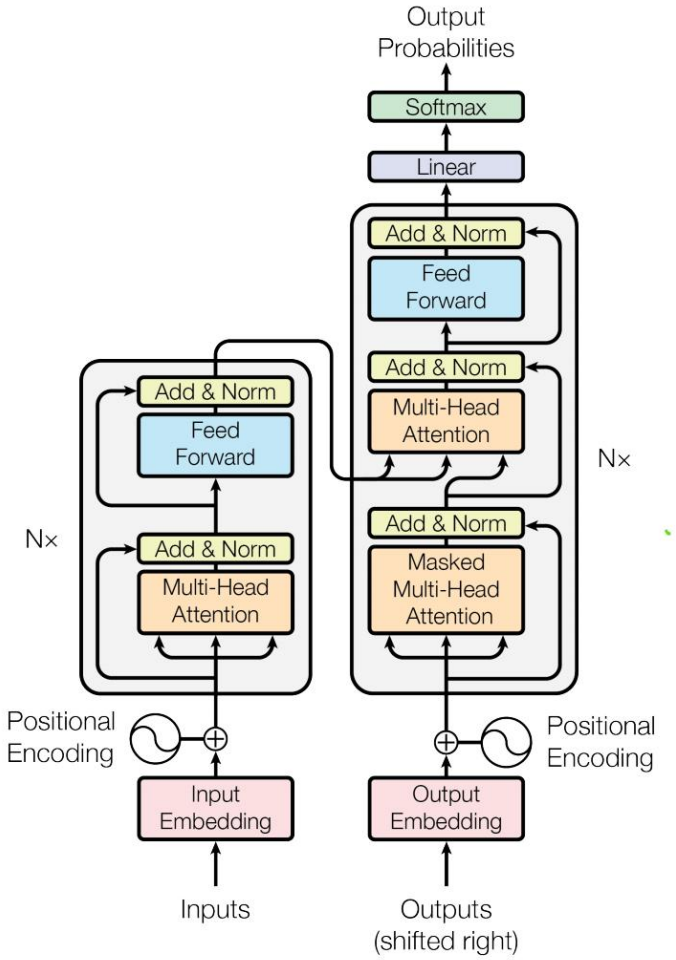
<BOS> I love learning

6.Transformer -- Encoder-Decoder



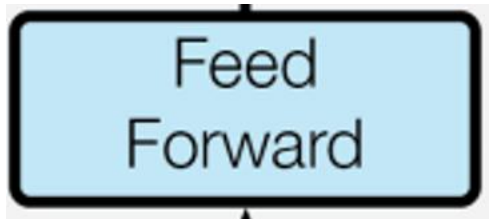
- 编码器（Encoder）：A
  负责「理解输入」，把整段输入文本加工成融合了全局上下文的特征向量
- 解码器（Decoder）：B
  负责「生成输出」，参考编码器的上下文特征，逐词生成目标文本

6.Transformer -- Encoder-Decoder





残差连接+归一化



FNN网络

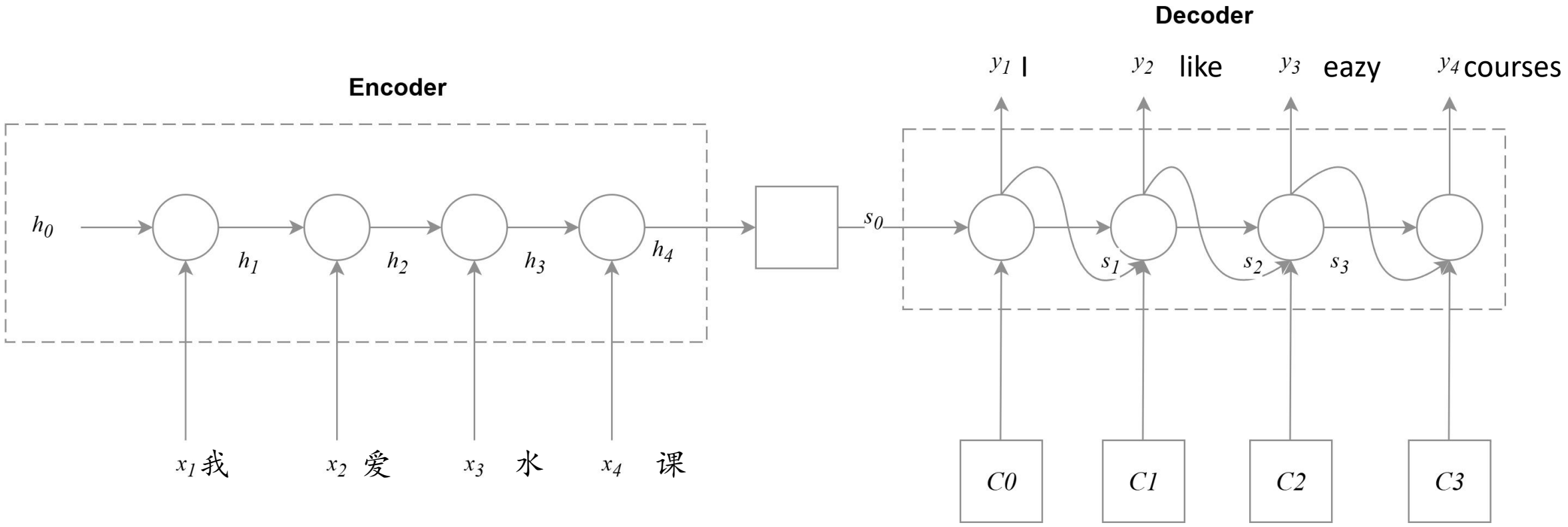


多头自注意力机制



掩码自注意力机制

4. Attention mechanism注意力机制

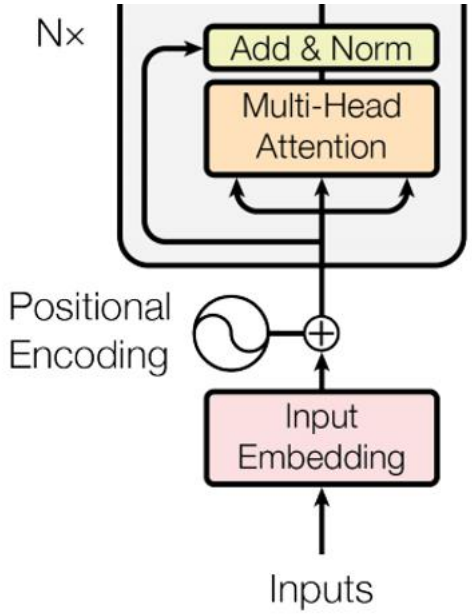


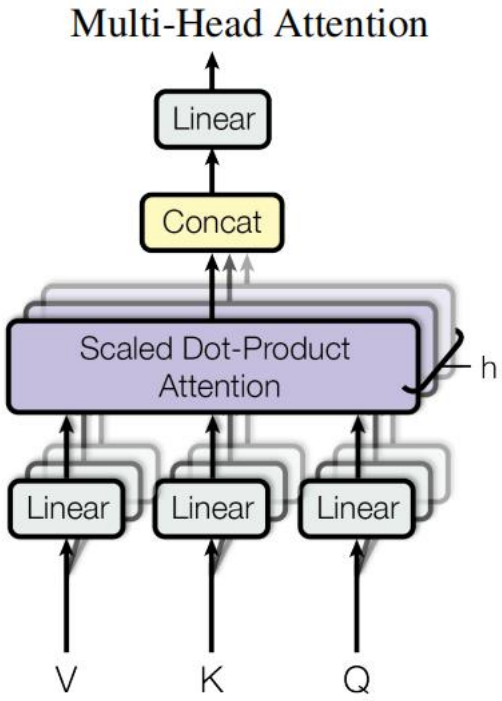
$$C_i = \sum_{j=1}^{n} \alpha_{ij} h_j$$

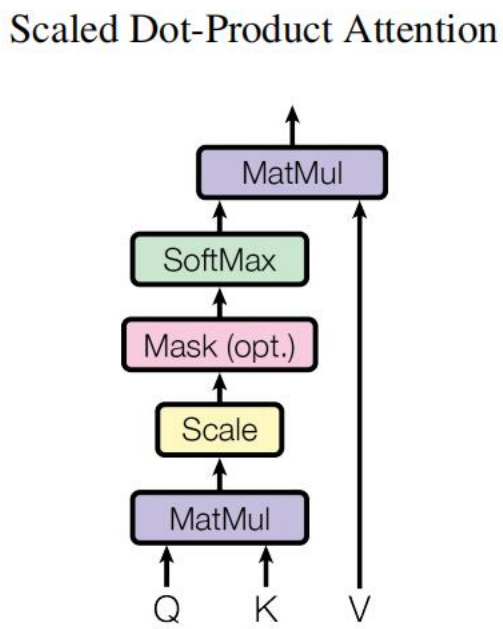
可训练参数

$$C_2 = 0.1h_1 + 0.1h_2 + 0.4h_3 + 0.4h_4$$

6.Transformer -- 单头自注意力机制

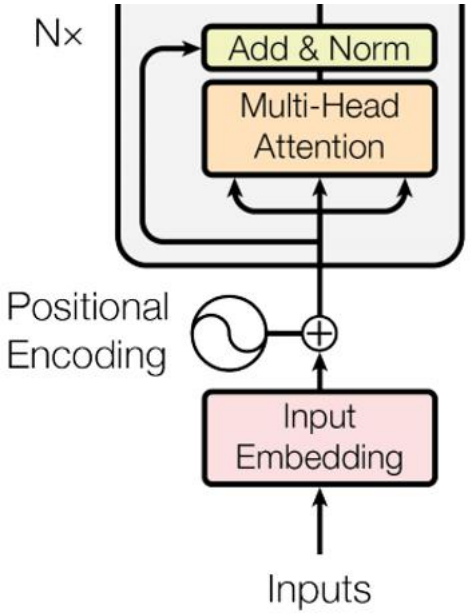


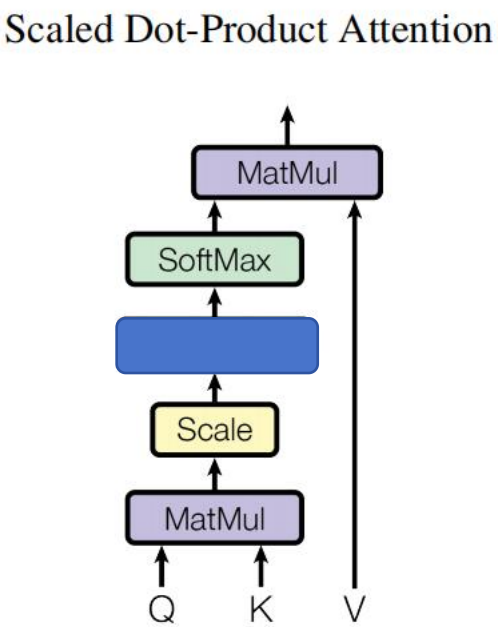




$$X = InputEmbedding(tokens) + PositionalEncoding$$
$$Q = XW^Q$$
$$K = XW^K$$
$$V = XW^V$$

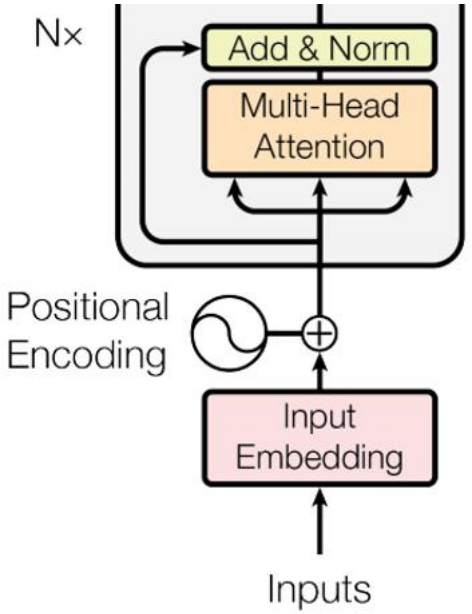
6.Transformer -- 单头自注意力机制

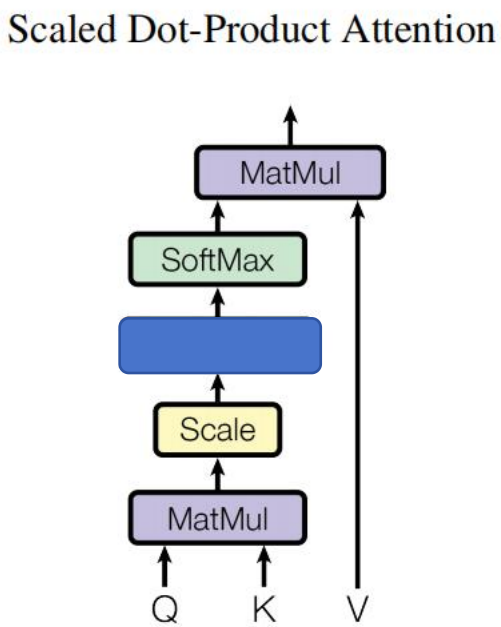




$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

6.Transformer -- 单头自注意力机制

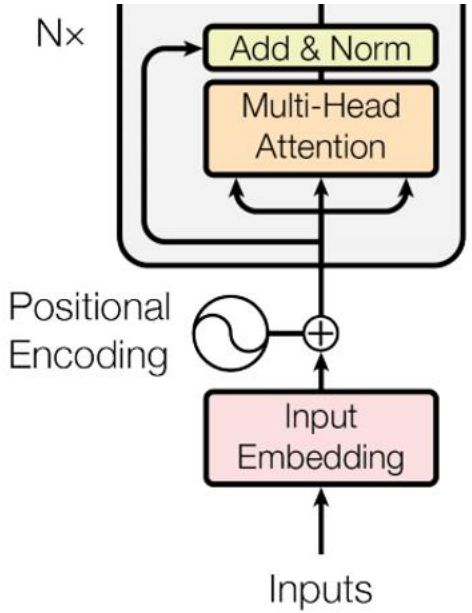


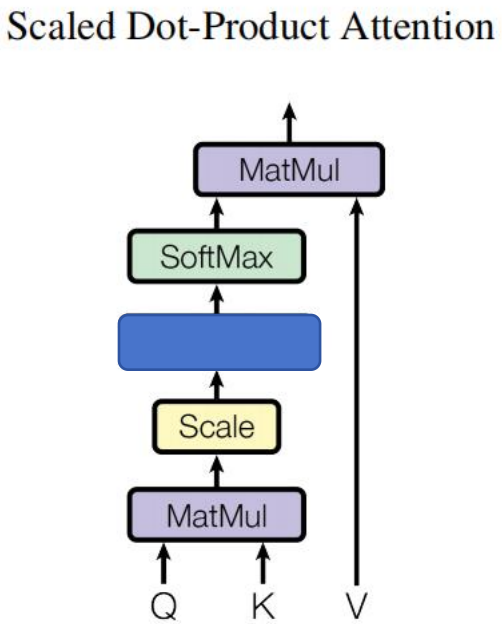


$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

V（Value，值向量）：真正携带我们最终要提取、聚合的信息本身。

6.Transformer -- 单头自注意力机制





$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Q(Query,查询向量):
代表当前 token 的检索意图。用来表达:
“当前这个词，想要去寻找序列中哪些 token 的信息。”

K（Key,键向量）:
代表序列里每个 token 可供匹配的特征。

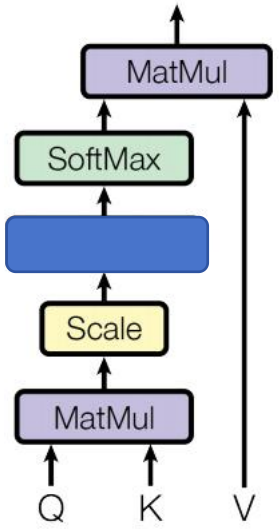
6.Transformer -- 单头自注意力机制

例句：桌子上放着苹果，它红红的

1. Q「它」：
寻找和“它”有关系的词语
2. K：

$$K = \begin{bmatrix} \text{桌子} & [\text{家具类实体, 硬质平面, 放置物品, 名词}] \\ \text{上} & [\text{无实体, 方位描述, 位置关系, 介词}] \\ \text{放着} & [\text{无实体, 动作状态, 放置行为, 动词}] \\ \text{苹果} & [\text{水果类实体, 红色圆形, 可食用, 名词}] \\ \text{它} & [\text{代词实体, 待描述外观, 待确定功能, 代词}] \\ \text{红红的} & [\text{无实体, 红色外观, 无功能, 形容词}] \end{bmatrix}$$

Scaled Dot-Product Attention

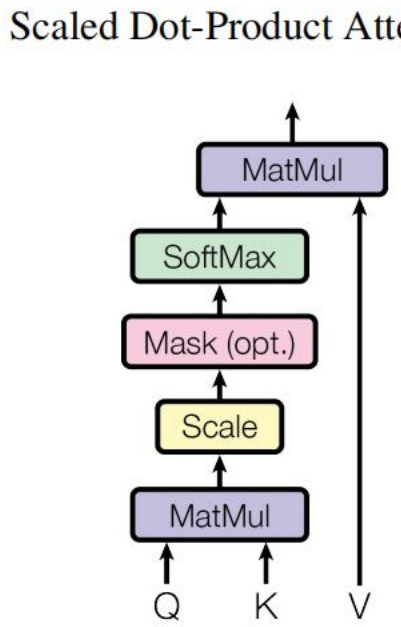


3. V 矩阵

$$V = \begin{bmatrix} \text{桌子} & [\text{家具类实物, 棕色方形平面, 用来摆放物品, 句中作主语/宾语}] \\ \text{上} & [\text{方位关系, 物体表面位置, 表示位置从属, 句中作方位状语}] \\ \text{放着} & [\text{放置动作, 静止存在状态, 表示物体位置状态, 句中作谓语}] \\ \text{苹果} & [\text{水果类实物, 红色圆形果皮, 可直接食用, 句中作指代对象}] \\ \text{它} & [\text{代词占位, 待指代的外观, 待指代的功能, 句中作主语}] \\ \text{红红的} & [\text{颜色属性, 红色外观描述, 修饰事物外观, 句中作补语}] \end{bmatrix}$$

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

6.Transformer -- 带掩码单头自注意力机制



只在训练中生效：

我喜欢水课

I like eazy courses

I like ____ xxxxxx

$$Attention(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V},\boldsymbol{M}) = softmax\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\top}}{\sqrt{d_k}} + \boldsymbol{M}\right)\boldsymbol{V}$$

掩码矩阵：

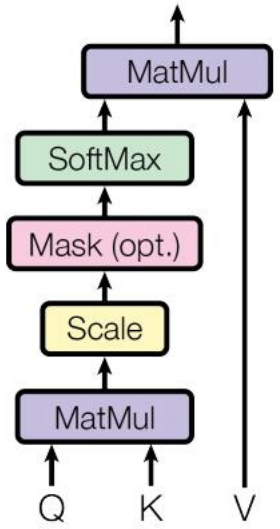
需要屏蔽的位置：取值为（或极大负数），经过 softmax 后对应注意力权重趋近于 0

保留的位置：取值为 0，不影响原始相似度分数

$$\mathrm{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

6.Transformer -- 带掩码单头自注意力机制

Scaled Dot-Product Attention

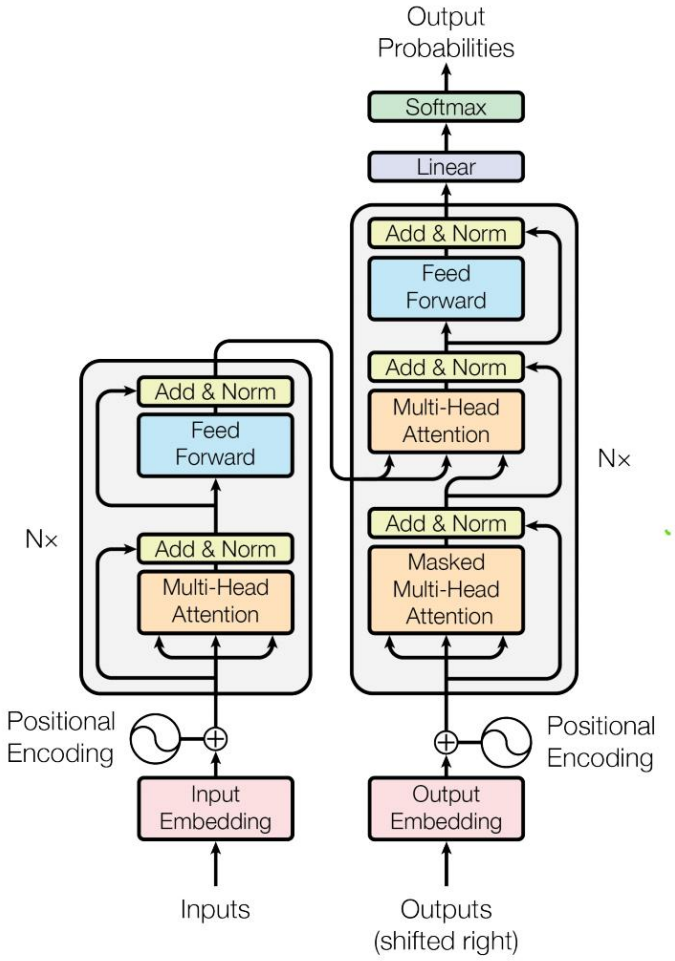


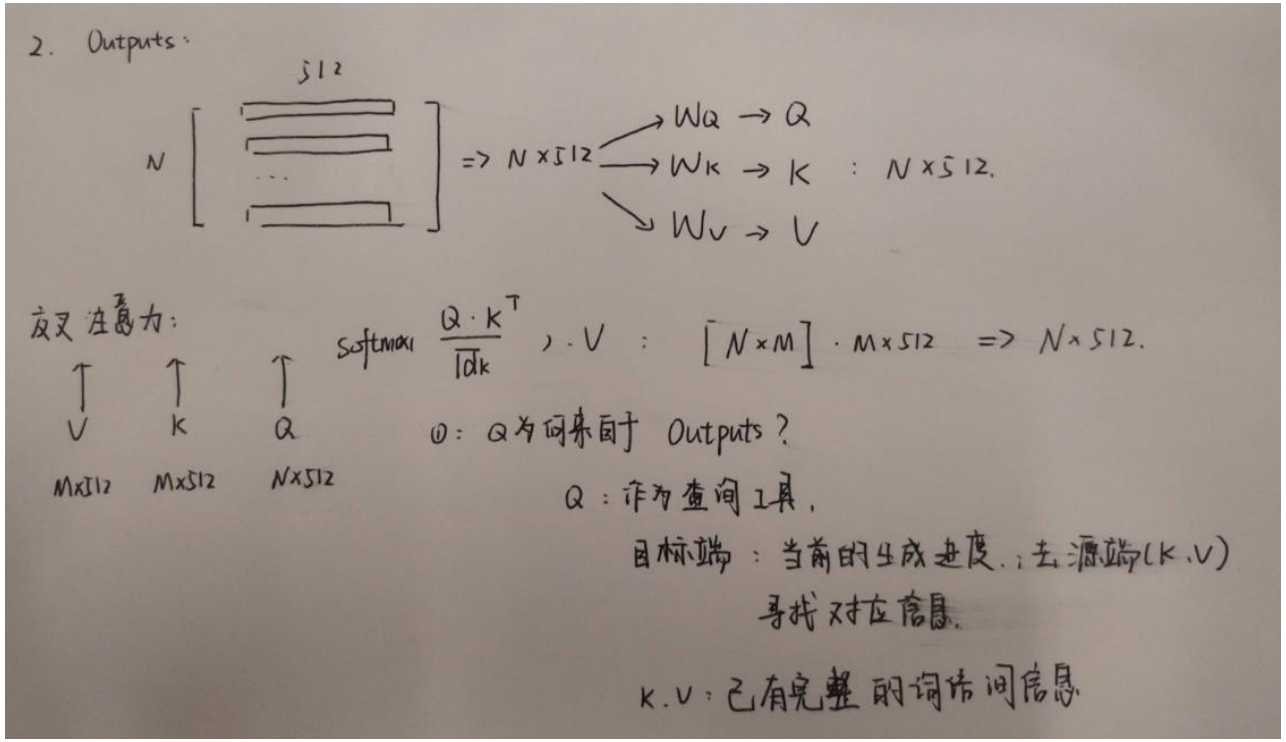
$$\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top = \begin{bmatrix} 3 & 1 & 1 & 2 \\ 2 & 2 & 1 & 0 \\ 1 & 1 & 3 & 2 \\ 2 & 0 & 2 & 3 \end{bmatrix} \quad \begin{bmatrix} 我 \\ 爱 \\ 水 \\ 课 \end{bmatrix} \quad \boldsymbol{M} = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

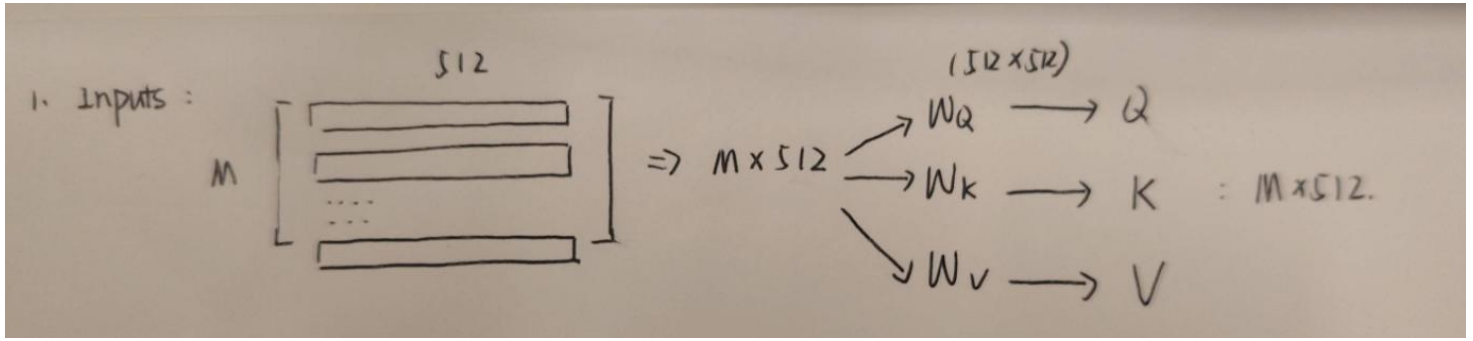
$$\boldsymbol{S} + \boldsymbol{M} = \begin{bmatrix} 3 & -\infty & -\infty & -\infty \\ 2 & 2 & -\infty & -\infty \\ 1 & 1 & 3 & -\infty \\ 2 & 0 & 2 & 3 \end{bmatrix}$$

$$\boldsymbol{A} = \mathrm{softmax}\left(\frac{\boldsymbol{S}+\boldsymbol{M}}{\sqrt{5}}\right) \approx \begin{bmatrix} 1.000 & 0 & 0 & 0 \\ 0.500 & 0.500 & 0 & 0 \\ 0.225 & 0.225 & 0.550 & 0 \\ 0.252 & 0.103 & 0.252 & 0.394 \end{bmatrix}$$

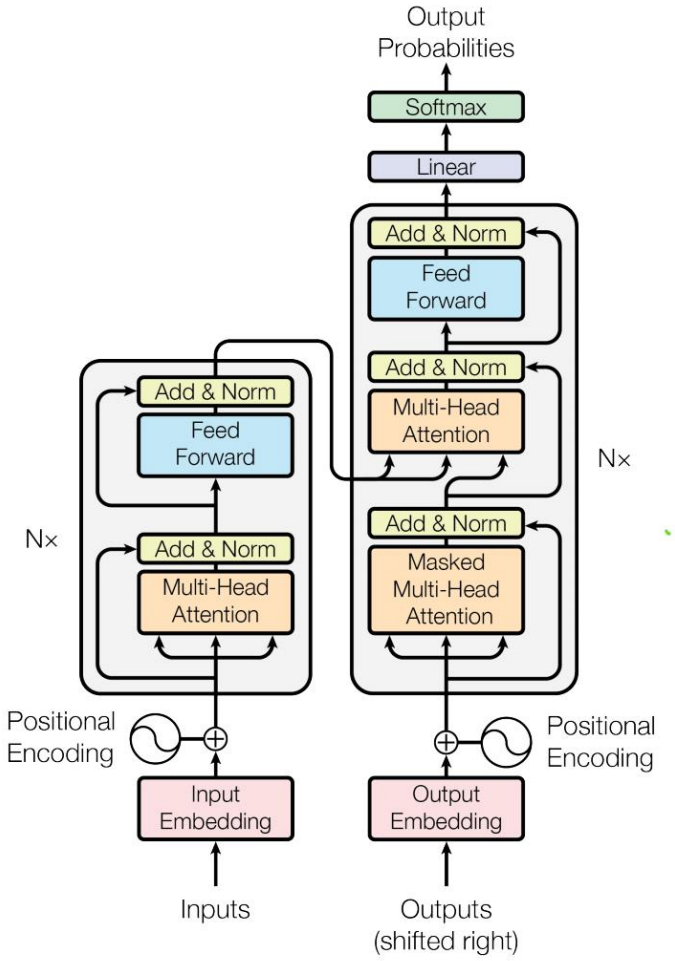
6.Transformer -- 交叉自注意力机制

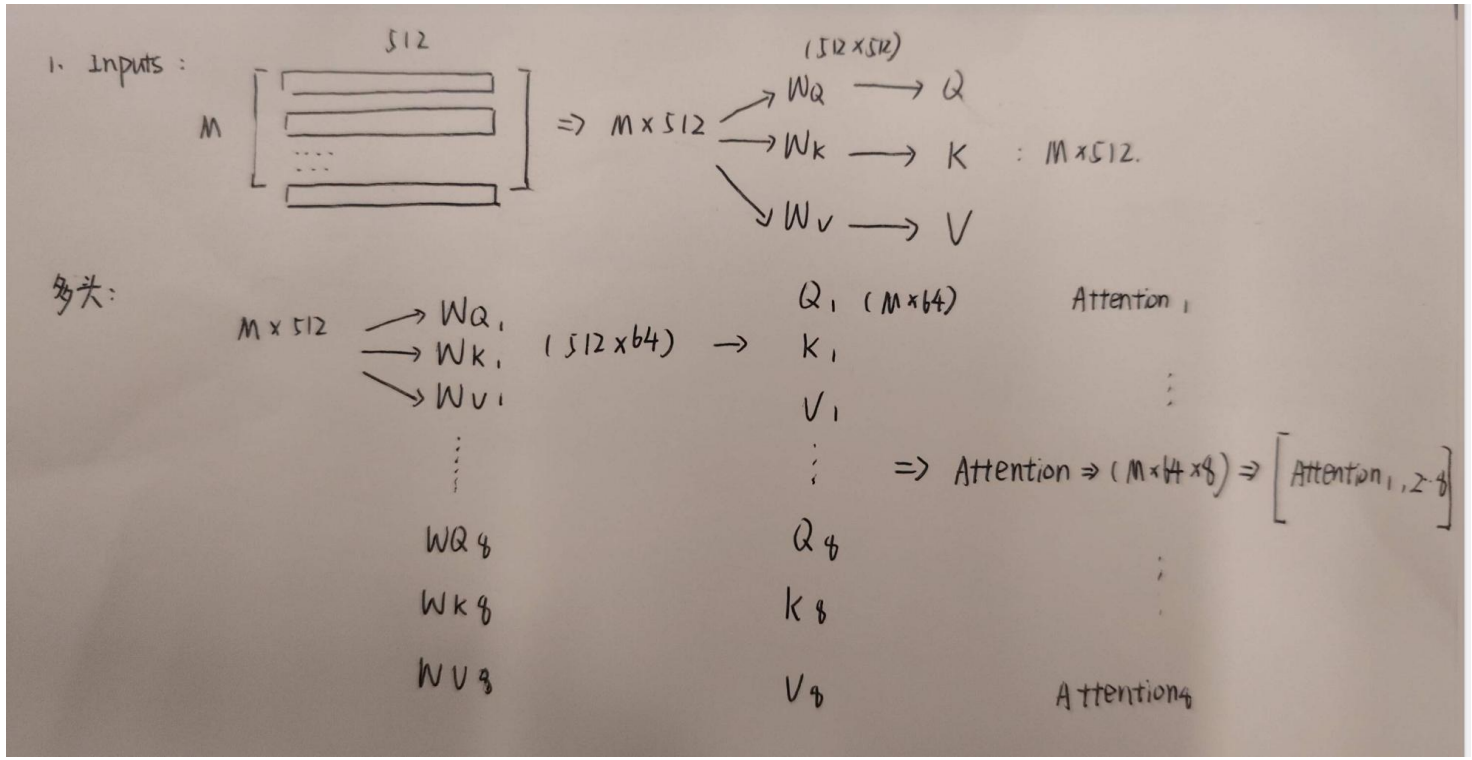




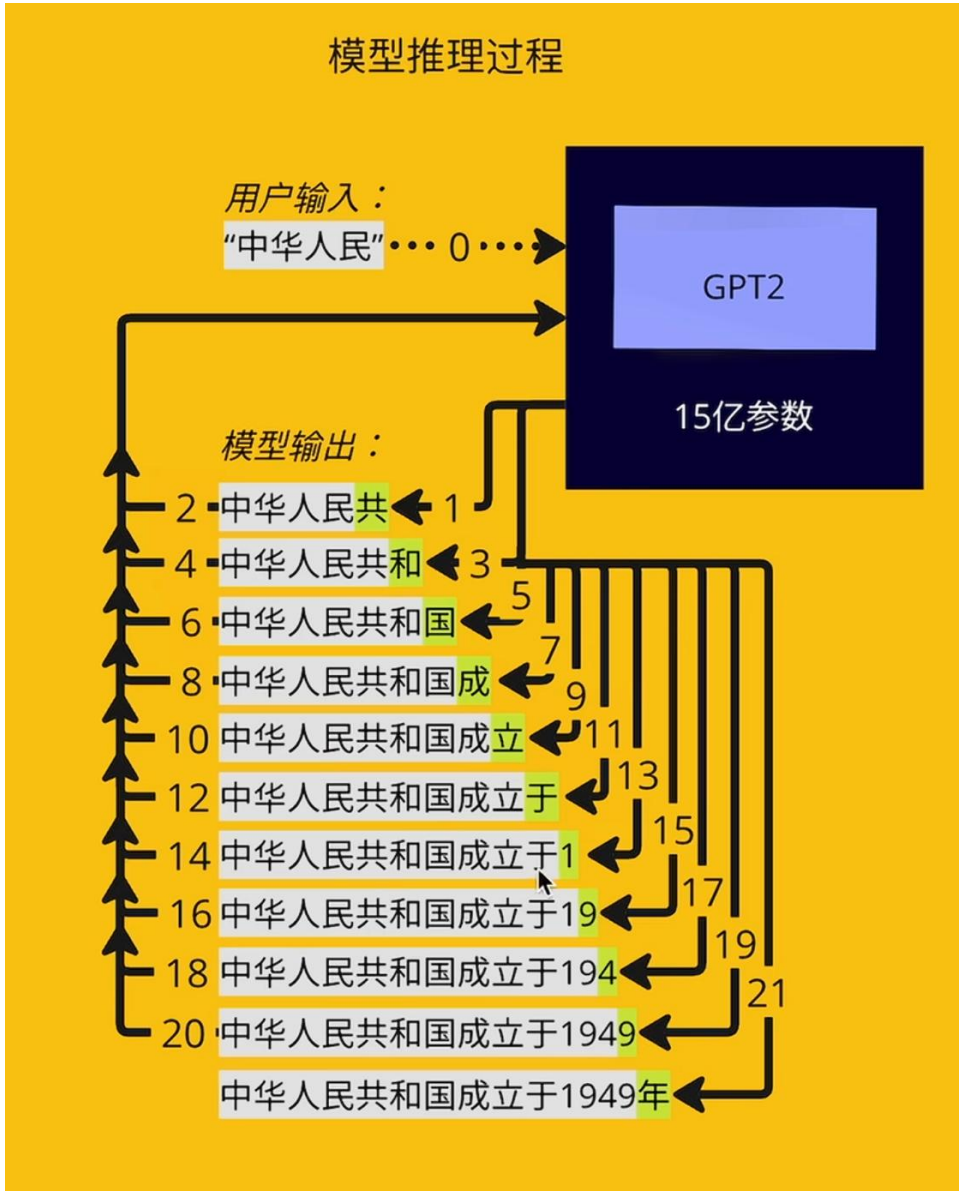


6.Transformer -- 多头注意力机制





KVcache



无 KV Cache：

需要输入完整上下文

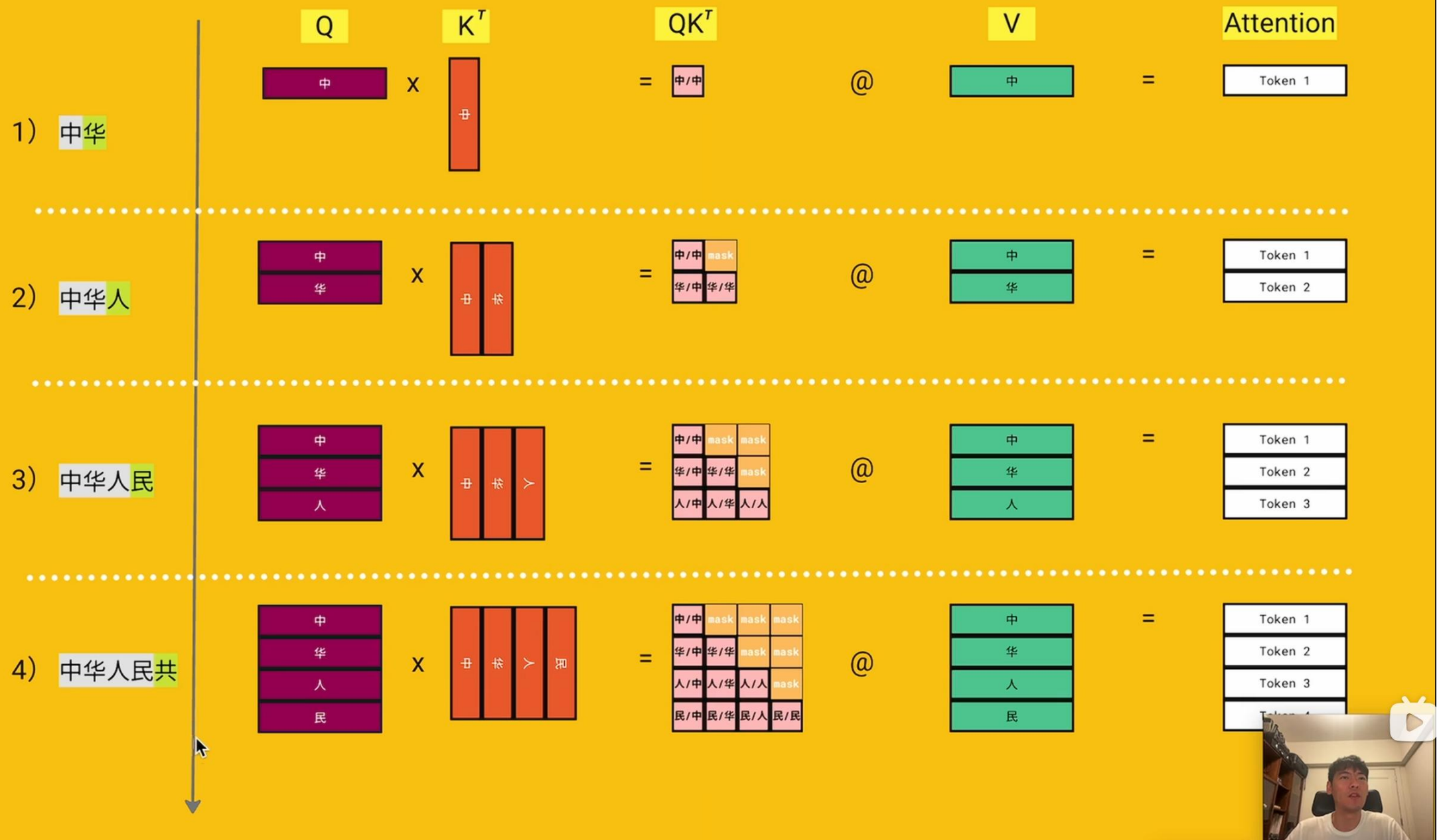
重新计算之前的所有K、V

有 KV Cache：

输入上一个生成的Token即可

LLM张老师 bilibili

不使用 KV Cache



LLM张老师 bilibili

使用 KV Cache

bilibili.com - 若要退出全屏，请从屏幕顶部向下轻扫或按 Esc

